

Day 4 Handout: Chatbot with Memory

AI Application Development Bootcamp · Chatbot with Memory + Project 1

Context vs Memory

Groq / LLaMA

Gradio

SQLite

Sentence Transformers

ChromaDB

MOODLE CODE SNIPPETS

All Python code files are provided on Moodle in **code snippets.zip**. In this handout, use the file name shown in each section, open that matching snippet, run it, and use the explanation here to understand the important lines.

Contents

1. Before You Run Any Code — Groq, API key, virtual environment, packages
2. Core Idea: Context vs Memory
3. Stateless Chatbots — terminal, `gr.Interface` , `gr.ChatInterface`
4. Basic Memory with a Python List
5. The Growing Context Problem — recent-history window, summary memory
6. Long-Term Memory with Files — Markdown, JSON, hybrid
7. Beyond Files — SQLite keyword search, semantic search with embeddings, ChromaDB
8. Suggested Testing Script
9. Full Build Sequence

1. Before You Run Any Code

All examples use Python and call a hosted LLM through [Groq](#). The full code files are on Moodle. The file names used below match the slides and the code snippets package.

What is Groq?

Groq gives your Python program access to a language model through an API. In these examples, the model is `llama-3.1-8b-instant`. It is useful for classroom demos because it is fast enough for interactive chatbot examples.

⚠️ GROQ IS ONLY THE MODEL PROVIDER

Groq does not automatically remember your users, store chat history, create your UI, or search your old messages. Those parts are built by *your application*. That is the entire point of today.

Create a Groq API key

1. Sign in to the Groq Console.
2. Create an API key.
3. Copy it once and keep it private. see the security warning below!

Create and activate a virtual environment

A [virtual environment](#) keeps the bootcamp packages separate from other Python projects.

Windows PowerShell

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

Install packages

For the first chatbot and Gradio examples:

terminal

```
pip install groq gradio
```

For semantic search and ChromaDB examples later:

terminal

```
pip install sentence-transformers numpy chromadb
```

💡 **SQLITE NEEDS NO PIP INSTALLATION**

`sqlite3` is part of Python's standard library, so you only need `import sqlite3`.

Set the API key

The snippets use:

Python

```
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

This means Python reads the key from an environment variable called `GROQ_API_KEY`.

Windows PowerShell

```
$env:GROQ_API_KEY="paste-your-api-key"
```

Use this when your terminal says
PowerShell .

Windows Command Prompt

```
set GROQ_API_KEY=paste-your-api-key-
```

Use `set` only in `cmd.exe` . It does not
work the same way in PowerShell.

macOS / Linux

```
export GROQ_API_KEY="paste-your-api-
```

This sets the key for the current terminal
session.

These commands are temporary. If you close the terminal, set the key again before running the snippets.

Alternative: `.env` file

If you prefer a project file, create `.env` in the same folder as your Python files:

`.env`

```
GROQ_API_KEY=paste-your-api-key-here
```

SECURITY WARNING

An API key is a credential, not configuration. Anyone who obtains it can make requests billed to your account. The most common exposure paths are:

1. **Hardcoded keys in source files.** Never write `api_key="gsk_..."` directly in a `.py` file. Code gets shared, submitted, and copied, the key travels with it. This is why every snippet in this handout reads the key from the `GROQ_API_KEY` environment variable instead.
2. **Shared output.** The key appears on screen at the moment you run the set-key command or open the `.env` file. Clear the terminal before recording, screen-sharing, or capturing anything for submission.
3. **Version control.** If you use Git, exclude the `.env` file before your first commit. Public repositories are continuously scanned for exposed credentials, and a key removed in a later commit remains recoverable from the repository history.

1. Create a `.gitignore` file in your project folder and add `.env` to it (see below)
2. Before every `git push`, verify with `git status` that `.env` is not staged

`.gitignore` – add this to your project

```
.env
.venv/
__pycache__/
.pyc
```

If a key is ever exposed, deleting the file is not enough, **revoke it in the Groq Console and generate a new one**. The exposed key remains valid until revoked.

Then install:

terminal

```
pip install python-dotenv
```

And add this before creating the Groq client:

Python

```
from dotenv import load_dotenv
load_dotenv()
```

How to run a snippet

Download and unzip `code snippets.zip` from Moodle. Open a terminal in that folder, activate your virtual environment, then run a file like this:

terminal

```
python stateless_chatbot.py
```

If the file launches a Gradio app, the terminal will print a local URL such as `http://127.0.0.1:7860` . Open that URL in your browser.

2. Core Idea: Context vs Memory

A chatbot does not automatically remember previous turns. Each API call only sees the context that your program sends in the `messages` list.

Term	Meaning	Example
LLM context	Information included in the current API request.	The <code>messages=[...]</code> list sent to Groq.
Application memory	Information saved outside the model and reused later.	A Python list, JSON file, Markdown file, SQLite database, or ChromaDB collection.

⚡ MAIN IDEA

Memory is an **application design decision**: what to store, where to store it, how to retrieve it, and how much to send back to the model.

3. Stateless Chatbots

3.1 Terminal Stateless Chatbot

`stateless_chatbot.py`

First chatbot. It runs in the terminal and sends only the latest user message to the model.

A stateless chatbot is the simplest possible version: user input goes in, the model replies, and nothing is saved for the next turn.

▶ Run it

terminal

```
python stateless_chatbot.py
```

Try this test:

```
You: My name is Alex.  
You: What is my name?
```

The bot may not know, because the second request only contains `What is my name?`. The first message was not sent again.

U

User

My name is Alex.

B

Bot

Nice to meet you, Alex!

U

User

What is my name?

B

Bot

I do not know your name from this message alone.

▶ Next

▶▶ Play All

↺ Reset

4 / 4 messages

The visible conversation above is what humans see. The model only sees what the code sends in `messages` . In this file, that is only the latest prompt.

Important line	What you need to know
<code>client = Groq(...)</code>	This creates the API client. The API key is not typed directly into the code; it is read from <code>GROQ_API_KEY</code> for safety.
<code>messages=[{"role":"user", ...}]</code>	This is the actual text sent to the model. Because it contains only one user message, the model receives no previous conversation.
<code>model="llama-3.1-8b-instant"</code>	This chooses which hosted model Groq should run for this request.
<code>while True</code>	This keeps the terminal chatbot running so users can ask more than one question. It does not create memory by itself.

3.2 Basic Gradio Interface

stateless_chatbot_webUI.py

Same stateless chatbot, but with a simple web page using `gr.Interface` .

Gradio helps you create a simple web UI without writing HTML or JavaScript. In this step, the chatbot logic is still the same. Only the input/output interface changes.

Terminal version	Gradio Interface version
User types into terminal using <code>input()</code> .	User types into a browser textbox.
Reply prints using <code>print()</code> .	Reply appears in the web output area.
No visible chat layout.	Simple input/output page.

terminal

```
python stateless_chatbot_webUI.py
```

Open the local Gradio URL. Test the same name example. It should still behave like a stateless chatbot because the model still receives only the latest message.

3.3 ChatInterface: Chat-Style UI, Still Stateless

stateless_chatbot_webUI_a.py

Uses `gr.ChatInterface` , which looks like a real chat UI and passes a `history` argument.

`gr.Interface` is a general-purpose UI wrapper. `gr.ChatInterface` is designed specifically for chatbot layouts. It shows chat bubbles and passes two values into your function: the latest `prompt` and the visible UI `history` .

⚠ **IMPORTANT DISTINCTION**

Seeing previous messages *on screen* does not mean the model remembers them. In this snippet, the function receives `history` , but the code intentionally ignores it. So the UI looks conversational, while the model call is still stateless.

Feature	<code>gr.Interface</code>	<code>gr.ChatInterface</code>
Main use	Any simple input/output function	Chatbot-style apps
Screen layout	Textbox + output box	Chat bubbles
Function input	Usually one input, e.g. <code>prompt</code>	<code>prompt</code> and <code>history</code>
Automatic model memory?	No	No. You still decide what to send to the LLM.

Important line	What you need to know
<code>def chat_bot(prompt, history)</code>	<code>ChatInterface</code> sends two values: the latest user message and the visible chat history. This is different from <code>gr.Interface</code> , which usually sends only the input value.
<code>gr.ChatInterface(...)</code>	This creates a chatbot-style UI with chat bubbles and a message history display.



Check Your Understanding_Section 3

If `ChatInterface` shows old messages on screen, why can the model still forget them?

- ☐ Because Groq deletes the model after every message
- ☒ Because UI history and model context are different; the code must send history in messages
- ☐ Because `ChatInterface` cannot pass history to Python

Check Answer

✓ Correct! `ChatInterface` displays previous messages and passes history into the Python function, but the LLM only sees the messages list that the code sends to Groq. In this snippet, history is ignored, so the model is still stateless.

4. Basic Memory with a Python List

`basicM_chatbot_webUI_a.py`

Stores recent conversation in a Python list called `memory` and sends it back to the model.

This is the first real memory version. The program appends the user message to a list, sends the list to the model, receives the reply, and then appends the assistant reply too.

Important line	What you need to know
<code>memory = []</code>	This list is the application memory. It exists while the Python program is running.
<code>memory.append({"role": "user", ...})</code>	The current user message is saved before calling the model, so it becomes part of the conversation context.
<code>messages=memory</code>	Now the model receives the full list, not only the latest prompt. This is what gives the bot short-term memory.
<code>memory.append({"role": "assistant", ...})</code>	The assistant reply is saved too. Without this, the model would not know what it already answered in previous turns.

 WHY SAVE THE ASSISTANT REPLY TOO?

The model needs to know both sides of the conversation. If you only save user messages, the history becomes incomplete and the bot may not know what it already answered.

Limitation: this memory is stored in RAM. If the program stops, the list is cleared.



Check Your Understanding_Section 4

Which line actually makes the LLM see the previous conversation in the basic memory version?

☐ `memory = []`

☐ `title="Basic Memory Chatbot"`

☐ `messages=memory`

Check Answer

5. The Growing Context Problem

The basic list memory version is useful because it proves the main idea: if we want the chatbot to remember, we must send previous messages back to the model. However, this approach has a serious limitation. Every new turn makes the `messages` list longer, and the whole list is sent again on every API call.

At the beginning this is fine. After a long conversation, the prompt becomes large, slower, more expensive, and less focused. The model may receive many old messages that are no longer useful for the current question.

Problem	What happens	Why we should care
Higher token usage	Every old message is sent again and again.	More tokens usually means higher cost when using paid APIs.
Slower responses	The model has more text to process before it can answer.	The chatbot feels less interactive.
Noisy context	Old or irrelevant details stay in the prompt.	The model may focus on the wrong information.
Context window limit	Each model has a maximum amount of text it can receive.	If the prompt becomes too large, the request may fail or important content must be removed.

⚠️ **TOKENS GROW WITH THE CONVERSATION**

The token count does not stay constant. A short conversation may only use a few hundred tokens, but a long conversation can grow to thousands or tens of thousands of tokens because the app keeps resending previous turns. This is why memory design is not just “save everything”.

Good memory means **selecting, compressing, or retrieving** the right information for the current turn.

🤖 **“BUT MODERN MODELS HAVE HUGE CONTEXT WINDOWS...”**

You might ask: modern models can accept hundreds of thousands of tokens, so why not just stuff the whole history in? Three reasons. First, you still **pay and wait** for every token on every call. Second, research shows models handle very long contexts imperfectly, information buried in the middle of a long prompt is often ignored, a failure known as the “*lost in the middle*” problem.^[7] Third, a context window resets between sessions; it is still not *persistence*. A big window makes memory design easier, not unnecessary.

5.1 Trade-offs before moving to long-term memory

Before writing memory to files or databases, we need to understand what each short-term strategy is good for. There is no single “best” memory method. Each one makes a different trade-off between simplicity, speed, detail, and reliability.

Strategy	What it sends to the model	Good for	Main trade-off
Full conversation history	Every user and assistant message so far.	Short demos and early prototypes, because the logic is very easy to understand.	The prompt grows forever. It can become slow, costly, and noisy.
Recent history window	Only the last few messages, for example the last 6 messages.	Chats where only the latest discussion matters, such as quick support or short Q&A.	Older facts disappear from the model's view, even if they are important.
Summary memory	A compressed summary of older turns plus recent messages.	Longer conversations where the overall direction matters more than exact wording.	Summaries are lossy. Once details are compressed, some exact information may be gone.

This is the point where long-term memory becomes useful. Short-term memory controls what the model sees during the current running session. Long-term memory stores information outside the current Python process so it can survive restarts, be searched later, and support larger applications.

5.2 Recent history window

basicM_chatbot_recentH_webUI_a.py
Keeps only the most recent messages, such as the last 6 messages.

A recent-history window is the simplest way to control context growth. The app still stores the conversation in `memory` , but before calling the model it slices the list and sends only the most recent messages.

This is useful when the chatbot only needs nearby context. For example, if the user asks a follow-up question about the immediately previous answer, the last few messages are usually enough. But if the user gave their name or project requirement at the start of the chat, that information may fall outside the window and be forgotten.

Important line	What you need to know
<code>memory.append({"role": "user", ...})</code>	The full list still stores each new user message while the app is running.
<code>recent_memory = memory[-6:]</code>	This creates a smaller list containing only the last 6 messages. The negative index means “count from the end of the list”.
<code>messages=recent_memory</code>	The model receives only the recent window, not the full conversation. This is the line that controls how much context is sent.
<code>memory.append({"role": "assistant", ...})</code>	The assistant reply is still saved so it can appear in future recent windows.

 WHEN THIS FAILS

If the user says “My hotel is near Shinjuku station” early in the conversation, and many turns pass after that, the message may fall outside the last 6 messages. Later, if the user asks “Where is my hotel?”, the model may not know because that fact was not sent in the current request.

5.3 Summary memory

basicM_chatbot_summary_webUI_a.py

Summarises older messages when memory becomes too long.

Summary memory solves a different problem. Instead of simply forgetting older turns, the app asks the LLM to compress the conversation into a shorter summary. The summary is then reused as context in future requests.

This is useful when the conversation has a storyline or ongoing task. For example, a project discussion may include goals, decisions, constraints, and pending tasks. The bot does not need every sentence exactly, but it should remember the overall direction.

Important line	What you need to know
<code>summary = ""</code>	This variable stores the compressed version of older conversation turns. It starts empty because there is no summary yet.
<code>text = "\n".join(...)</code>	This converts the current message list into plain text so it can be given to the model for summarisation.
<code>Summarize this conversation briefly</code>	The app uses the LLM itself to create the summary. This is a common pattern: one model call can help manage memory for later model calls.
<code>summary = response.choices[0].message.content</code>	The model's summary is saved in the <code>summary</code> variable.
<code>memory = []</code>	After summarising, the detailed messages are cleared. This keeps the prompt small, but exact details may be lost.
<code>if summary:</code>	Before each answer, the app checks whether a summary exists. If yes, it adds it as system context.
<code>if len(memory) > 8</code>	This is the trigger. Once the recent message list becomes longer than 8 items, the app summarises it.

 IMPORTANT LIMITATION

Summarisation is not perfect memory. It is **compressed** memory. If the summary does not include a small detail, that detail may be unavailable later. This is why summaries are good for broad context, but not ideal for exact facts such as IDs, dates, allergies, or grading rules.

6. Long-Term Memory with Files

The previous methods keep memory inside the running Python program. If the program stops, list-based memory disappears. Long-term memory means saving information outside the program, usually to a file or database, so the chatbot can load it again later.

However, storage alone is not enough. A chatbot also needs a **retrieval strategy**. Retrieval means deciding which saved information should be sent back to the model for the current user question. If we save everything but always send everything, we are back to the growing context problem.

6.1 Markdown memory

MD_chatbot_webUI_a.py

Appends conversation turns into `memory.md` .

Markdown memory is the simplest persistent memory format. The chatbot writes each user and assistant turn into a plain text file. When the app runs again, it can read that file and include the previous memory in the prompt.

You can open `memory.md` and directly see what was saved. It is also useful for small personal assistants or prototype bots where human readability is more important than search performance.

Important line	What you need to know
<code>FILE = "memory.md"</code>	This defines where the memory is stored. The file becomes the chatbot's persistent memory.
<code>with open(FILE, "r")</code>	Opens the file for reading so the previous memory can be loaded before generating a reply.
<code>except: return ""</code>	If the file does not exist yet, the bot starts with empty memory instead of crashing.
<code>with open(FILE, "a")</code>	"a" means append mode. New conversation turns are added to the end of the file instead of replacing older memory.
<code>Previous memory:\n{memory_text}</code>	The saved text is placed in the system message. This is how external memory becomes visible to the model in the current request.

⚠️ **TRADE-OFF**

Markdown is easy to read but weak for retrieval. The snippet loads the whole file, so it is not ideal when memory becomes large or when you only need one relevant part of the old conversation.

6.2 JSON memory

Json_chatbot_webUI_a.py

Stores conversation as a structured list of message dictionaries in `chat.json` .

JSON is more structured than Markdown. Instead of saving the chat as free text, it saves the conversation as a list of dictionaries. This matches the format expected by the Groq `messages` parameter.

This means the app can load the JSON file and send the stored list directly to the model. It is cleaner than Markdown for programmatic use because each message still has a `role` and `content` .

Important line	What you need to know
<code>FILE = "chat.json"</code>	This is the file where structured chat history is stored.
<code>json.load(f)</code>	Reads the previous message list from the JSON file and converts it back into Python data.
<code>except: return []</code>	If the file does not exist yet or cannot be loaded, the bot starts with an empty list.
<code>memory.append({"role": "user", ...})</code>	Adds the current user message in the same structure expected by the LLM API.
<code>messages=memory</code>	Sends the loaded conversation history plus the current message to the model.
<code>json.dump(memory, f)</code>	Writes the updated list back to disk so the memory survives after the app closes.

💡 WHEN JSON IS USEFUL

JSON is a good next step after list memory because it keeps the same message structure but makes it persistent. It is still best for small demos, because the whole file is loaded and saved each time.

6.3 Hybrid JSON + Markdown memory

Json+MD_chatbot_webUI_a.py

Uses JSON for dynamic chat history and Markdown for fixed profile or skills knowledge.

The hybrid version separates two different kinds of memory. The JSON file stores **dynamic conversation history**: what the user and assistant said during the chat. The Markdown file stores more stable **background knowledge**, such as user profile notes, project context, or skills information.

This is a more realistic design because not all memory changes at the same speed. A conversation changes every turn, but profile information or project instructions may stay the same for a longer time.

Important line	What you need to know
JSON_FILE = "chat.json"	This file stores the changing conversation history.
MD_FILE = "test_skills.md"	This file stores more stable background knowledge.
chat_memory = load_chat_memory()	Loads previous conversation turns from JSON.
profile_memory = load_profile()	Loads profile or skills text from Markdown.
messages = [{"role": "system", ...}]	Starts the prompt with stable profile knowledge as system context.
messages.extend(chat_memory)	Adds the dynamic chat history after the profile information.

⚡ WHY THIS PATTERN MATTERS

A useful chatbot often combines different memory types: fixed instructions, user preferences, recent chat history, and retrieved facts. Keeping them separate makes the design easier to control.

7. Why Move Beyond File-Based Memory?

So far, we have explored several memory approaches: Python list memory, recent-history memory, summary memory, Markdown memory, JSON memory, and a hybrid JSON + Markdown approach. These are good stepping stones because they are simple, visible, and easy to understand.

However, as soon as the chatbot grows beyond a small classroom demo, these methods start to show clear trade-offs. They can store information, but they are not always good at searching, filtering, or selecting only the most relevant memory for the current question.

Memory method	Good for	Main trade-off
Python list	Understanding the basic idea of memory during one running session.	Memory disappears when the program stops.
Recent-history window	Keeping the prompt short by sending only the latest messages.	Older but important facts may be forgotten.
Summary memory	Compressing long conversations into a shorter form.	Summaries are useful, but they can lose exact details.
Markdown file	Human-readable notes and simple memory logs.	Easy to read, but difficult to search in a structured way.
JSON file	Saving chat history in a structured format.	The whole file is usually loaded and saved again each time.
JSON + Markdown hybrid	Separating dynamic chat history from fixed profile or skills knowledge.	Still file-based, so retrieval becomes harder as memory grows.

The key issue is not just *saving* memory. The harder problem is *retrieving the right memory at the right time*. A chatbot should not blindly send everything it has ever stored. That would make the prompt larger, slower, more expensive, and more distracting for the model.

⚠️ IMPORTANT TRADE-OFF

File-based memory is excellent for learning and small prototypes, but it does not scale well when we need to search many messages, filter by role or time, support multiple users, or retrieve only the most relevant past information.

A common next step is to store memory in a database. Instead of treating the whole chat history as one large text file, a database stores each message as a separate record. This allows the application to insert new messages, search existing ones, and retrieve only the rows that may be useful for the current user question.

For this reason, we now move to **SQLite**. SQLite is a lightweight database that comes with Python, so it is a practical first database for chatbot memory. It helps us move from “save everything in a file” toward “store memory in a structured and searchable way.”

7.1 Long-Term Memory with SQLite

`SQLite_chatbot_webUI_a.py`

Stores each message as a row in a SQLite table and retrieves matching memory using keyword search.

SQLite is the next step when plain files become too limited. JSON and Markdown are easy to understand, but they are not designed for flexible searching. SQLite stores memory in rows and columns, so the app can query only the records it needs.

Think of the difference this way: with JSON, the app usually loads the whole file, edits it, and saves the whole file again. With SQLite, the app can insert one new row or search for matching rows without manually reading and rewriting everything.

File memory	SQLite memory
Usually loads a whole file.	Can query selected rows.
Simple to inspect manually.	Better for structured data and larger memory.
Weak search support.	Supports SQL queries such as filtering by keyword, role, or time.
Good for small prototypes.	Better bridge toward real applications.

In this snippet, SQLite is still doing **keyword search**, not semantic search. That means it looks for matching text using `LIKE` . If the saved memory contains “Tokyo”, searching for “Tokyo” can find it. But if the user asks “Where am I travelling in Japan?”, keyword search may fail unless the exact words overlap.

Important line	What you need to know
<code>sqlite3.connect("memory_sqlite.db", check_same_thread=False)</code>	Opens a local SQLite database file. <code>check_same_thread=False</code> is used because Gradio may call the function from a different thread.
<code>cursor = conn.cursor()</code>	The cursor is used to send SQL commands to the database.
<code>CREATE TABLE IF NOT EXISTS history (...)</code>	Creates the memory table if it does not already exist. This table stores each message as one row.
<code>role TEXT</code>	Stores whether the message came from the <code>user</code> or the <code>assistant</code> .
<code>content TEXT</code>	Stores the actual message text.
<code>INSERT INTO history (role, content) VALUES (?, ?)</code>	Saves one new memory row. The <code>?</code> placeholders safely insert values without building SQL by string concatenation.
<code>WHERE content LIKE ?</code>	Searches for rows where the content contains the keyword pattern.
<code>(f"%{keyword}%",)</code>	The <code>%</code> symbols mean “match anything before or after this keyword”. For example, searching <code>Tokyo</code> can match “My flight is to Tokyo next Friday.”
<code>return "\n".join(...)</code>	Combines all matching rows into one memory block that can be sent back to the model.

 THE KEY LIMITATION

SQLite gives us structured storage, but `LIKE` search only checks text patterns. It does not understand *meaning*. This is why the next step introduces embeddings and semantic search.

7.2 Semantic Search with Embeddings

SQLite_Semantic_chatbot_webUI_a.py

Stores text embeddings in SQLite and manually ranks memories using cosine similarity.

Keyword search asks: “Did the same word appear?” Semantic search asks: “Is the *meaning* similar?” This is important because users often rephrase their questions.

For example, suppose the saved memory says: “*My hotel is near Shinjuku station.*” Later, the user asks: “*Where am I staying in Tokyo?*” Keyword search may not find the memory because the words are different. Semantic search has a better chance because the meaning is related.

An embedding is a list of numbers that represents the meaning of a text. Similar sentences should produce vectors that are close to each other. The app can compare the user’s current question vector with saved memory vectors and retrieve the closest matches.

Keyword search	Semantic search
Matches exact words or text patterns.	Compares meaning using vectors.
May miss synonyms such as “car” and “automobile”.	Can retrieve related wording if the embeddings are close.
Simple and fast to teach.	Requires an embedding model and similarity calculation.
Good for exact terms, names, IDs, and keywords.	Good for rephrased questions and meaning-based retrieval.

The SQLite semantic snippet is educational because it shows the internal mechanism directly. Instead of letting a vector database handle everything, the code stores embeddings in SQLite and manually calculates similarity scores.

Important line	What you need to know
<code>SentenceTransformer("all-MiniLM-L6-v2")</code>	Loads a local embedding model. This model converts text into vectors that represent meaning.
<code>embedding = embed_model.encode(content).tolist()</code>	Converts each saved message into a vector before storing it. <code>tolist()</code> changes it into a normal Python list so it can be saved.
<code>json.dumps(embedding)</code>	SQLite does not directly store Python lists, so the vector is converted into JSON text before saving.
<code>query_embedding = embed_model.encode(prompt)</code>	Converts the current user question into a vector, so it can be compared with saved memory vectors.
<code>SELECT content, embedding FROM history</code>	Loads saved messages and their embeddings from the database.
<code>np.array(json.loads(embedding_text))</code>	Converts the stored JSON text back into a NumPy array for mathematical comparison.
<code>np.dot(query_embedding, memory_embedding) / (...)</code>	Calculates cosine similarity. A higher score means the saved memory is more similar in meaning to the current prompt.
<code>results.sort(reverse=True)</code>	Sorts memories from most relevant to least relevant based on similarity score.
<code>top_results = results[:3]</code>	Keeps only the top 3 memories. This avoids sending every old message back to the model.

 NOTICE

This version is not just “SQLite with extra comments”. It changes the **retrieval logic**. The chatbot no longer searches only for matching words. It searches for stored messages that are close in meaning to the user’s current question.



Check Your Understanding_Section 7

Memory contains *"My hotel is near Shinjuku station."* The user asks *"Where am I staying in Tokyo?"* Why can keyword search fail here while semantic search succeeds?

- ☐ Keyword search only works on JSON files, not SQLite
- ☐ The memory is too old, so SQLite deleted the row
- ☐ No exact words overlap, so LIKE finds nothing; embeddings can still be close because the meanings are related

Check Answer

7.3 ChromaDB: A Vector Database

ChromaDB_chatbot_Semantic_webUI_a.py

Uses ChromaDB to store embeddings and retrieve top matching memories automatically.

[ChromaDB](#) is introduced after the SQLite semantic version because now you know what problem it solves. In the SQLite semantic code, the app manually stores vectors, loads all rows, converts JSON back into arrays, calculates cosine similarity, sorts the scores, and selects the top results. That is useful for learning, but it becomes messy as the memory grows.

A vector database is designed specifically for this kind of task. It stores documents with their embeddings and provides a query function to retrieve the closest matches. This makes the application code cleaner and closer to how real retrieval systems are built.

SQLite semantic version	ChromaDB version
Stores embeddings manually as JSON text.	Stores documents, embeddings, and metadata together.
Loads every stored memory and loops through them.	Uses the vector database query function to retrieve matches.
Manually calculates cosine similarity.	Handles similarity search for you.
Best for understanding how semantic retrieval works.	Better preview of production-style vector retrieval.

Important line	What you need to know
<code>chromadb.PersistentClient(path="chroma_memory")</code>	Creates a local ChromaDB database folder. Because it is persistent, stored memory can survive app restarts.
<code>get_or_create_collection("history")</code>	Creates or loads a collection. A collection is like a named storage area for related memories.
<code>collection.add(...)</code>	Saves the memory text, its embedding, metadata such as role, and an ID into ChromaDB.
<code>metadatas=[{"role": role}]</code>	Stores extra information about the memory. Here, it records whether the message came from the user or assistant.
<code>ids=[str(collection.count())]</code>	Gives each stored item an ID. This simple demo uses the current count as the next ID.
<code>collection.count() == 0</code>	Checks whether memory exists before searching. This prevents querying an empty collection.
<code>collection.query(query_embeddings=[...], n_results=3)</code>	Asks ChromaDB to return the top 3 memories closest to the current prompt embedding.
<code>results["documents"][0]</code>	Extracts the retrieved memory texts from the query result.

BRIDGE TO DAY 9

Notice the pattern here: *embed* → *store* → *search by meaning* → *send only the best matches to the model*. On Day 9 you will apply exactly the same pattern to **documents** instead of chat messages — that technique is called RAG (Retrieval-Augmented Generation). Memory retrieval and document retrieval are the same core idea.

8. Suggested Testing Script

Use these facts to test whether memory and retrieval are working:

```
My flight is to Tokyo next Friday.  
My hotel is near Shinjuku station.  
I am allergic to peanuts.  
My favorite programming language is Python.  
I am building a chatbot for learning AI.
```

Then ask:

```
What is my flight?  
Where is my hotel?  
Where am I traveling in Japan?  
Do I have any food restrictions?  
What coding language do I like most?  
What project am I working on?
```

For keyword search, questions may fail when the exact wording is different. For semantic search, the bot has a better chance of retrieving the correct memory even when the wording changes.

💡 FRESH-START TIP

Persistent memory files survive restarts: `memory.md` , `chat.json` , `memory_sqlite.db` , and the `chroma_memory` folder. If the bot “remembers” strange things from earlier testing, delete the memory file or folder to start a clean test. This is a feature, not a bug — it proves long-term memory works.

9. Full Build Sequence

Order	File to run	Main idea
1	<code>stateless_chatbot.py</code>	Terminal chatbot; no memory.
2	<code>stateless_chatbot_webUI.py</code>	Simple Gradio <code>Interface</code> .
3	<code>stateless_chatbot_webUI_a.py</code>	Chat-style <code>ChatInterface</code> ; still stateless.
4	<code>basicM_chatbot_webUI_a.py</code>	Short-term memory using a Python list.
5	<code>basicM_chatbot_recentH_webUI_a.py</code>	Recent-history memory window.
6	<code>basicM_chatbot_summary_webUI_a.py</code>	Summary memory.
7	<code>MD_chatbot_webUI_a.py</code>	Persistent Markdown memory.
8	<code>Json_chatbot_webUI_a.py</code>	Persistent JSON memory.
9	<code>Json+MD_chatbot_webUI_a.py</code>	Hybrid profile + chat memory.
10	<code>SQLite_chatbot_webUI_a.py</code>	Searchable SQLite memory with keyword search.
11	<code>SQLite_Semantic_chatbot_webUI_a.py</code>	Semantic memory with embeddings and manual similarity.
12	<code>ChromaDB_chatbot_Semantic_webUI_a.py</code>	Semantic memory using a vector database.
13	Your Lab Chatbot	Build your own chatbot using any memory architecture so far!

10. How Real Memory Systems Work — and What Is Still Hard

Every snippet in this handout stores *raw conversation turns*. Production assistants — such as the memory features in ChatGPT and Claude, or open-source memory layers like MemO and Letta (formerly MemGPT) — go several steps further. This section previews those ideas so you can recognise them in real systems and, if you finish the lab early, try them yourself.

10.1 Fact extraction: store facts, not transcripts

Instead of saving every message, the app can ask the LLM to extract only the facts worth remembering — for example, turning a long message into "User is allergic to peanuts." The store stays small, and retrieval returns clean facts instead of chat fragments. This combines the summarisation idea from Section 5.3 with the retrieval idea from Section 7, and it is the core pattern behind modern memory layers like MemO.^[9]

Real fact-extraction systems do more than *add*. When a new fact arrives, the system compares it with what is already stored and decides between three operations:

Operation	When it happens	Example
ADD	The fact is new.	"User is learning Japanese" → stored as a new memory.
UPDATE	The fact changes an existing memory.	"I moved from Tokyo to Osaka" → the old location memory is replaced, not kept alongside the new one.
DELETE	The fact invalidates an existing memory.	"Actually I'm not allergic to peanuts, that was my sister" → the allergy memory is removed.

 THE STALENESS PROBLEM

If your chatbot only ever ADDs, its memory fills with contradictions: "user lives in Tokyo" *and* "user lives in Osaka". Retrieval may then surface the outdated fact, and the bot confidently answers with old information. Deciding when to update or delete is one of the genuinely hard problems in memory design — an LLM has to judge whether two facts conflict.

10.2 Types of memory in agent research

Research on LLM agents often borrows vocabulary from cognitive science to classify what an assistant remembers:^[10]



Episodic memory

What happened. Specific events and conversations: “On Tuesday the user asked about visa requirements.” Our raw-transcript snippets (JSON, SQLite) are episodic memory.



Semantic memory

What is true. Stable facts and preferences distilled from events: “User is allergic to peanuts.” Fact extraction converts episodic memory into semantic memory.



Procedural memory

How to behave. Instructions and learned routines: “Always answer this user in formal English with code examples.” Our Markdown profile file in Section 6.3 is a simple form of this.

Systems like Letta/MemGPT treat the context window like a computer’s RAM and the external store like a hard disk: the agent itself decides what to page in, what to page out, and can even edit its own memory blocks.^[8] That is a more advanced version of exactly what you built today with the summary + retrieval snippets.

10.3 Forgetting and relevance scoring

Remembering everything forever is not the goal — retrieving the *right* memory is. A well-known research design (the “generative agents” paper) scores each memory on three signals before deciding what to retrieve.^[11]

- **Relevance** — how semantically close is the memory to the current question? (This is the cosine similarity you computed in Section 7.2.)
- **Recency** — newer memories usually matter more; older ones decay.
- **Importance** — an allergy matters more than small talk, even if both are old.

Your ChromaDB snippet already implements the first signal. Adding a timestamp column and a simple importance score is a realistic extension for your lab chatbot.

10.4 Privacy and security of stored memory

Notice what our testing script saves: an allergy, travel plans, a hotel location. Memory files like `chat.json` and `memory_sqlite.db` are plain, unencrypted files on disk. In a real application you must decide who can read stored memory, how a user can view and delete it (a legal requirement

under regulations such as GDPR and Singapore’s PDPA), and whether sensitive facts should be stored at all.

MEMORY IS ALSO AN ATTACK SURFACE

Persistent memory creates a new security risk: **memory poisoning**. If an attacker can get a malicious instruction saved into memory (for example through a pasted document or a manipulated conversation), the bot may retrieve and obey it in every future session. Treat stored memory as untrusted input, and treat it with the same care as your API key.

10.5 How do you know your memory works? Evaluation

“It seems to remember” is not a measurement. Recent benchmarks such as **LongMemEval** and **LoCoMo** test assistants on long, multi-session conversations and check abilities like recalling facts across sessions, reasoning over time (“what did I say *before* I changed jobs?”), updating knowledge, and correctly *abstaining* when the answer was never mentioned.^[12] Even strong commercial assistants lose significant accuracy on these tests — memory is far from a solved problem. For your lab chatbot, the testing script in Section 8 is a miniature version of the same idea: fixed facts in, rephrased questions out, and you count how many the bot gets right.

LAB IDEA (IF YOU FINISH EARLY)

Try the fact-extraction pattern in your own lab chatbot: after each reply, make one extra LLM call that outputs zero or more short facts, and store only those facts in your memory backend. Bonus challenge: before adding a fact, retrieve similar stored facts and ask the LLM whether the new fact should ADD, UPDATE, or DELETE.

WHAT YOU SHOULD UNDERSTAND BY THE END

A memory-aware chatbot is not just a model call. It is an **application design**. You decide what to store, where to store it, how to retrieve it, how to keep it correct over time, and how much to send back to the model.

References

Use these as starting points only. You do not need to read them fully before the lab.

- [1]** Groq API Quickstart. console.groq.com/docs/quickstart
- [2]** Gradio — Interface and ChatInterface Documentation. gradio.app/docs
- [3]** Python `json` module documentation. docs.python.org/3/library/json.html
- [4]** Python `sqlite3` module documentation. docs.python.org/3/library/sqlite3.html
- [5]** Sentence Transformers documentation. sbert.net
- [6]** ChromaDB documentation. docs.trychroma.com
- [7]** Liu, N. et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. arxiv.org/abs/2307.03172
- [8]** Packer, C. et al. (2023). MemGPT: Towards LLMs as Operating Systems. arxiv.org/abs/2310.08560 · Letta project: letta.com
- [9]** Chhikara, P. et al. (2025). MemO: Building Production-Ready AI Agents with Scalable Long-Term Memory. arxiv.org/abs/2504.19413 · memO.ai
- [10]** Zhang, Z. et al. (2024). A Survey on the Memory Mechanism of Large Language Model based Agents. arxiv.org/abs/2404.13501
- [11]** Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arxiv.org/abs/2304.03442
- [12]** Wu, D. et al. (2024). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. arxiv.org/abs/2410.10813